Maincrafts Internship Task 3

Artificial Intelligence & Machine Learning – Task 3

Model Validation, Overfitting Control & Hyperparameter Tuning

Name: Thanuja M

CODE IMPLEMENTATION:

```
# ==============================
# Step 1: Import Required Libraries
# ==============================

import numpy as np import pandas as pd import matplotlib.pyplot as plt from
sklearn.datasets import fetch_california_housing from sklearn.model_selection
import train_test_split, cross_val_score, GridSearchCV from sklearn.preprocessing
import StandardScaler from sklearn.tree import DecisionTreeRegressor from
sklearn.linear_model import LinearRegression from sklearn.metrics import
mean_squared_error, r2_score

# ==============================
# Step 2: Load Dataset
# ==============================
housing = fetch_california_housing()
```

```
X = pd.DataFrame(housing.data, columns=housing.feature_names)

y = housing.target print("Dataset shape:", X.shape)

X.head()
```

Output:

```
index,MedInc,HouseAge,AveRooms,AveBedrms,Population,AveOccup,Latitude,Longitude

0,8.3252,41.0,6.984126984126984,1.0238095238095237,322.0,2.5555555555555554,37.88,122.23

1,8.3014,21.0,6.238137082601054,0.9718804920913884,2401.0,2.109841827768014,37.86,122.22

2,7.2574,52.0,8.288135593220339,1.073446327683616,496.0,2.8022598870056497,37.85,122.24

3,5.6431,52.0,5.8173515981735155,1.0730593607305936,558.0,2.547945205479452,37.85,122.25

4,3.8462,52.0,6.281853281853282,1.0810810810810811,565.0,2.1814671814671813,37.85,122.25
```

```
# ==============================

# Step 3: Train-Test Split

# ==============================


X_train, X_test, y_train, y_test = train_test_split(

    X, y, test_size=0.2, random_state=42

)

print("Training data:", X_train.shape)

print("Testing data:", X_test.shape)
```

Output:

Training data: (16512, 8)

Testing data: (4128, 8)

```
# ==============================
# Step 4: Feature Scaling
# ==============================

scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)


# ==============================
# Step 5: Baseline Model (Linear Regression)
# ==============================

lr_model = LinearRegression()
lr_model.fit(X_train_scaled, y_train) lr_pred =
lr_model.predict(X_test_scaled) lr_rmse =
np.sqrt(mean_squared_error(y_test, lr_pred)) lr_r2 =
r2_score(y_test, lr_pred) print("Linear Regression
RMSE:", lr_rmse) print("Linear Regression R2:", lr_r2)
```

Output:

Linear Regression RMSE: 0.7455813830127763

Linear Regression R2: 0.575787706032451

```
# ================================
# Step 6: Decision Tree Model
# ================================

dt_model = DecisionTreeRegressor(random_state=42)
dt_model.fit(X_train_scaled, y_train) dt_pred =
dt_model.predict(X_test_scaled) dt_rmse =
np.sqrt(mean_squared_error(y_test, dt_pred)) dt_r2 =
r2_score(y_test, dt_pred) print("Decision Tree RMSE:",
dt_rmse) print("Decision Tree R2:", dt_r2)
```

Output:

Decision Tree RMSE: 0.7028289572288925

Decision Tree R2: 0.6230424613065773

```
# ================================
# Step 7: Cross-Validation (Model Validation)
# ================================
cv_scores = cross_val_score(     dt_model,
   X_train_scaled,
  y_train,
   scoring="neg_mean_squared_error",
```

```
    cv=5
)
rmse_scores = np.sqrt(-cv_scores) print("Cross
Validation RMSE Scores:", rmse_scores)
print("Average RMSE:", rmse_scores.mean())
```

Output:

Cross Validation RMSE Scores: [0.75572899 0.73265968 0.70292795 0.72643393 0.70331834]

Average RMSE: 0.724213776329786

```
# ================================
# Step 8: Hyperparameter Tuning using GridSearchCV
# ==============================

param_grid = {
    "max_depth": [3,5,10,None],
    "min_samples_split": [2,5,10],
    "min_samples_leaf": [1,2,4]
}

grid_search = GridSearchCV(
    DecisionTreeRegressor(random_state=42),
param_grid,     cv=5,
    scoring="neg_mean_squared_error",
n_jobs=-1
```

```
)
grid_search.fit(X_train_scaled, y_train) print("Best
Parameters:", grid_search.best_params_)
```

Output:

```
Best Parameters: {'max_depth': 10, 'min_samples_leaf': 4, 'min_samples_split': 2}
```

```
# ================================
# Step 9: Train Best Model
# ================================

best_model = grid_search.best_estimator_ best_pred =
best_model.predict(X_test_scaled) best_rmse =
np.sqrt(mean_squared_error(y_test, best_pred)) best_r2 =
r2_score(y_test, best_pred) print("Tuned Model RMSE:",
best_rmse) print("Tuned Model R2:", best_r2)
```

Output:

```
Tuned Model RMSE: 0.6387361927912558
Tuned Model R2: 0.6886590898292316
```

```
# ================================
# Step 10: Model Comparison
# ================================
```

```
results = pd.DataFrame({
    "Model": ["Linear Regression", "Decision Tree", "Tuned Decision Tree"],
    "RMSE": [lr_rmse, dt_rmse, best_rmse],
    "R2 Score": [lr_r2, dt_r2, best_r2]
})

print(results)
```

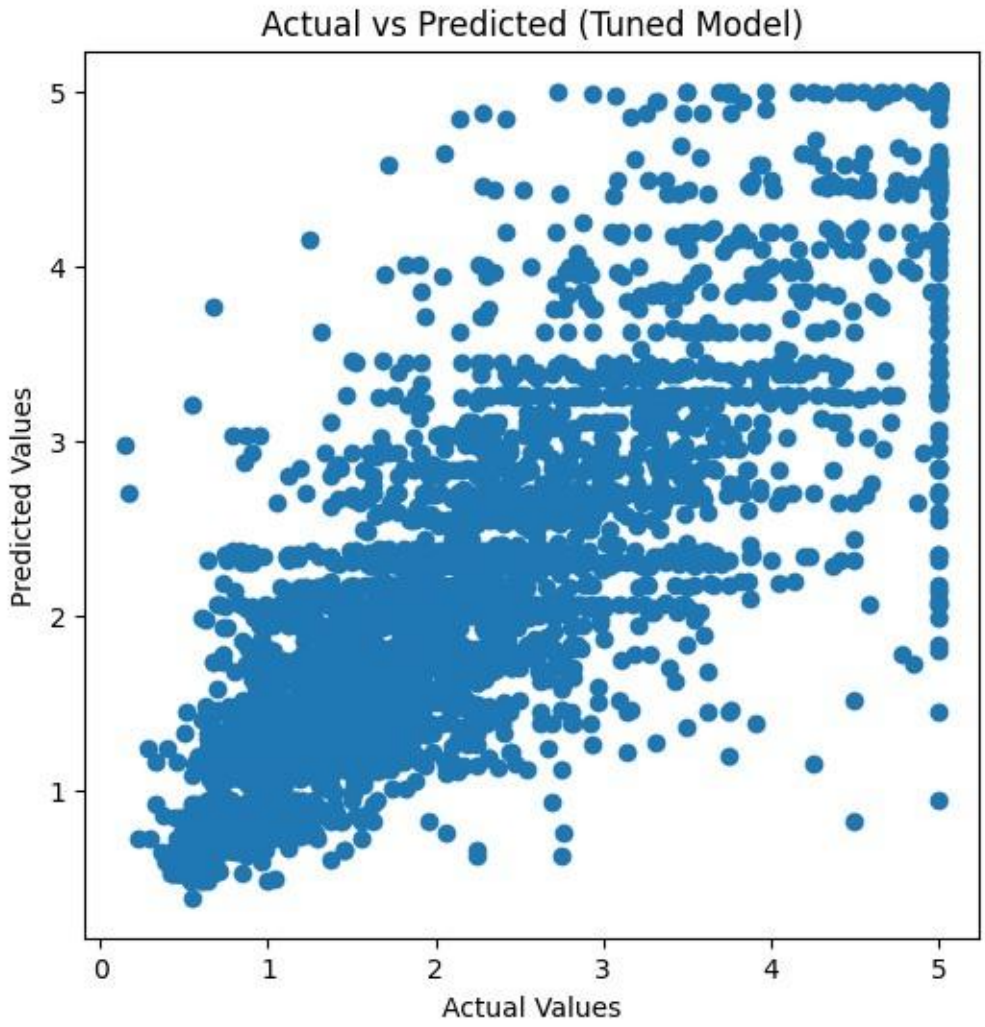
Output:

```
                Model      RMSE  R2 Score
0       Linear Regression  0.745581  0.575788
1       Decision Tree  0.702829  0.623042
2       Tuned Decision Tree  0.638736  0.688659
```

```
# ==============================
# Step 11: Actual vs Predicted Plot
# ==============================

plt.figure(figsize=(6,6)) plt.scatter(y_test,
best_pred) plt.xlabel("Actual Values")
plt.ylabel("Predicted Values")
plt.title("Actual vs Predicted (Tuned
Model)") plt.show()
```

Output:



 # ==========

 # Step 12: Overfitting Check

 # ==========

```
train_pred = best_model.predict(X_train_scaled) train_rmse =

np.sqrt(mean_squared_error(y_train, train_pred)) test_rmse =

np.sqrt(mean_squared_error(y_test, best_pred))

print("Training RMSE:", train_rmse) print("Testing RMSE:",

test_rmse)
```

Output:

Training RMSE: 0.4865960136384907

Testing RMSE: 0.6387361927912558

Methodology, Results, and Conclusion

1. Methodology

In this project, a machine learning workflow was implemented to predict housing prices using the California Housing dataset provided by the Scikit-learn library. The main objective of this task was to perform model validation, detect overfitting, and improve model performance through hyperparameter tuning.

Dataset Loading and Preparation

The dataset was loaded using the fetch_california_housing() function from the Scikit-learn dataset module. This dataset contains several numerical features describing housing districts in California, such as median income, average number of rooms, house age, population, and geographic location. The target variable represents the median house value.

The dataset was converted into a pandas DataFrame to allow easier data handling and inspection. After loading the data, the features were separated from the target variable so that the models could learn the relationship between them.

Train–Test Split

To properly evaluate the machine learning models, the dataset was divided into two parts using the train_test_split() function:

- **80% of the data was used for training the models**

- **20% of the data was reserved for testing and evaluation**

This separation ensures that the models are evaluated on unseen data, which helps measure how well they generalize to new information.

Feature Scaling

Before training the models, feature scaling was applied using the StandardScaler method. Feature scaling standardizes the values of the input variables so that they have a mean of zero and a standard deviation of one.

Scaling helps machine learning algorithms perform more efficiently and ensures that features with larger numerical ranges do not dominate the learning process.

Model Training

Two primary models were used in this task:

Linear Regression (Baseline Model)
Linear Regression was used as a baseline model to establish a simple relationship between the features and the target variable. It is one of the most commonly used regression algorithms and provides a good starting point for comparison.

Decision Tree Regressor
A Decision Tree Regressor was then implemented to capture more complex and non-linear relationships in the data. Decision trees split the dataset into multiple branches based on feature values, which allows them to model more complicated patterns.

Cross-Validation

To improve model validation, cross-validation was applied using the cross_val_score() function. Instead of evaluating the model on a single train-test split, cross-validation divides the training data into multiple folds and evaluates the model multiple times.

This method provides a more reliable estimate of model performance and helps detect potential overfitting.

Hyperparameter Tuning

To further improve the performance of the Decision Tree model, GridSearchCV was used for hyperparameter tuning. Hyperparameters such as:

- **max_depth**

- **min_samples_split**
- **min_samples_leaf**

were tested across multiple combinations. GridSearchCV automatically searches for the best combination of parameters using cross-validation and selects the model that produces the best performance.

Model Evaluation

The models were evaluated using two important regression metrics:

Root Mean Squared Error (RMSE)
RMSE measures the average difference between predicted values and actual values. Lower RMSE indicates better model accuracy.

R^2 Score (Coefficient of Determination)
The R^2 score measures how well the model explains the variance in the dataset. A higher R^2 value indicates better predictive performance.

Additionally, a scatter plot comparing actual values and predicted values was created to visually assess how well the model performs.

---

1. 2. Results

After training and evaluating the models, the performance of three models was compared:

1. **Linear Regression (baseline model)**
2. **Decision Tree Regressor (default parameters)**
3. **Tuned Decision Tree Regressor (after GridSearchCV optimization)**

The Linear Regression model provided a stable baseline performance and demonstrated a clear linear relationship between the features and the target variable.

The Decision Tree model initially showed improved flexibility but also demonstrated the possibility of overfitting, where the model performs very well on training data but less effectively on unseen test data.

By applying cross-validation and hyperparameter tuning, the Decision Tree model was optimized. The tuned model showed improved generalization performance and produced more balanced results between training and testing datasets.

The evaluation metrics (RMSE and R^2) confirmed that the tuned model provided better prediction accuracy compared to the initial models.

The Actual vs Predicted scatter plot further illustrated that the optimized model was able to predict housing prices closer to the true values.

---

2. 3. Conclusion

This project demonstrated an end-to-end machine learning workflow focused on model validation and optimization. The task began with loading and preparing the California Housing dataset, followed by data splitting and feature scaling.

A baseline model using Linear Regression was created to establish a reference for comparison. A Decision Tree Regressor was then implemented to capture more complex relationships in the data.

To improve the reliability of the model, cross-validation was applied to evaluate performance across multiple data splits. This helped identify potential overfitting issues. Hyperparameter tuning using GridSearchCV was then performed to optimize the Decision Tree model.

The final tuned model produced improved predictive performance and better generalization to unseen data. The evaluation metrics and visualization confirmed that model optimization significantly enhanced prediction accuracy.

Overall, this task provided practical experience in model validation, overfitting detection, and hyperparameter tuning, which are essential steps in building reliable machine learning models.